

# CO4-AT3

Student ID: 192421331

Student Name: JEEVANESHWARAN S

Assigned Question: Q14

## Q14. Assembly Line Scheduling vs Job Sequencing (DP vs Greedy)

Scheduling problems are used in industries to decide which job should be processed first and how work should be assigned to available resources. Two important approaches are Assembly Line Scheduling using Dynamic Programming (DP) and Job Sequencing with Deadlines using a Greedy Algorithm. Assembly Line Scheduling selects the best path through multiple assembly lines, while Job Sequencing selects and orders profitable jobs before their deadlines.

### 1. Assembly Line Scheduling Using Dynamic Programming

Assume there are 2 assembly lines and 4 stations.

#### 2. Dynamic Programming States

$T1[i]$  = minimum time to reach station  $i$  on Line 1.

$T2[i]$  = minimum time to reach station  $i$  on Line 2.

$T1[1] = \text{Entry1} + a_{1,1}$

$T2[1] = \text{Entry2} + a_{2,1}$

$T1[i] = \min(T1[i-1] + a_{1,i}, T2[i-1] + t_{2,i-1} + a_{1,i})$

$T2[i] = \min(T2[i-1] + a_{2,i}, T1[i-1] + t_{1,i-1} + a_{2,i})$

Minimum Time =  $\min(T1[n] + \text{Exit1}, T2[n] + \text{Exit2})$

#### 3. Step-by-Step Calculation

Station 1:  $T1[1] = 2 + 4 = 6$ ;  $T2[1] = 4 + 5 = 9$ .

Station 2:  $T1[2] = \min(6+5, 9+3+5) = 11$ ;  $T2[2] = \min(9+4, 6+2+4) = 12$ .

Station 3:  $T1[3] = \min(11+3, 12+1+3) = 14$ ;  $T2[3] = \min(12+6, 11+2+6) = 18$ .

Station 4:  $T1[4] = \min(14+2, 18+2+2) = 16$ ;  $T2[4] = \min(18+3, 14+1+3) = 18$ .

After exit times: Line 1 =  $16+3 = 19$ ; Line 2 =  $18+2 = 20$ .

Minimum Processing Time = 19 units.

#### 4. Algorithm for Assembly Line Scheduling

AssemblyLine(a, t, entry, exit, n)

$T1[1] = \text{entry1} + a_{1[1]}$

$T2[1] = \text{entry2} + a_{2[1]}$

for  $i = 2$  to  $n$ :

$T1[i] = \min(T1[i-1] + a_{1[i]},$   
 $T2[i-1] + t_{2[i-1]} + a_{1[i]})$

$T2[i] = \min(T2[i-1] + a_{2[i]},$   
 $T1[i-1] + t_{1[i-1]} + a_{2[i]})$

answer =  $\min(T1[n] + \text{exit1},$   
 $T2[n] + \text{exit2})$

```
return answer
```

## 5. Correctness of Assembly Line DP

At every station, the DP method considers both possibilities: continuing on the same line or transferring from the other line. For each station it stores the minimum time needed to reach that state. Because the optimal solution contains optimal preceding states, the optimal substructure property holds and the final value is globally optimal.

## 6. Job Sequencing with Deadlines Using Greedy Algorithm

Consider the following jobs, where each job requires one unit of processing time:

Job | Deadline | Profit

J1 | 2 | 100

J2 | 1 | 50

J3 | 2 | 40

J4 | 1 | 70

J5 | 3 | 60

The objective is to maximize total profit while completing selected jobs before their deadlines.

## 7. Greedy Strategy

1. Sort jobs by decreasing profit.
2. Select the highest-profit job.
3. Place it in the latest available slot before its deadline.
4. Continue until all jobs have been considered or no valid slot remains.

Sorted jobs: J1(2,100), J4(1,70), J5(3,60), J2(1,50), J3(2,40).

J1 → Slot 2

J4 → Slot 1

J5 → Slot 3

J2 → rejected because Slot 1 is occupied

J3 → rejected because Slots 1 and 2 are occupied.

Selected jobs: J4, J1, J5

Total Profit:  $70 + 100 + 60 = 230$ .

## 8. Job Sequencing Algorithm

```
JobSequencing(jobs):
```

```
    sort jobs by decreasing profit
    create slots based on maximum deadline
```

```
    for each job:
        for slot = deadline down to 1:
            if slot is empty:
                schedule job
                break
```

```
    return selected jobs and total profit
```

## 9. Correctness of Greedy Method

The greedy strategy selects the highest-profit job first and places it as late as possible. Placing a job in the latest available slot preserves earlier slots for jobs having smaller deadlines. For the standard unit-time Job Sequencing with Deadlines problem, this greedy strategy is optimal.

## **10. Comparison of DP and Greedy**

Assembly Line Scheduling uses Dynamic Programming to minimize total processing time. Job Sequencing uses a Greedy strategy to maximize total profit. DP evaluates alternative previous states, while Greedy commits to a locally best choice based on profit. Both can be optimal when their underlying problem assumptions are satisfied.

## **11. Processing Time Comparison**

The assembly-line example produces a minimum processing time of 19 units. The job-sequencing example produces a maximum profit of 230. These numerical values should not be compared directly because they represent different objectives. The useful comparison is how DP and Greedy make scheduling decisions.

## **12. Time and Space Complexity**

Assembly Line Scheduling: Time =  $O(n)$  for two lines. Space =  $O(n)$ , or  $O(1)$  using rolling states.

Job Sequencing: Sorting =  $O(n \log n)$ ; basic slot search =  $O(nD)$ ; total =  $O(n \log n + nD)$ , where  $D$  is the maximum deadline. Slot space =  $O(D)$ .

## **13. Real-World Applicability**

Assembly Line Scheduling is applicable to automobile manufacturing, electronics production, food processing, and multi-stage manufacturing. It can reduce production time, waiting time, transfer cost, and bottlenecks.

Job Sequencing is applicable to cloud computing, CPU scheduling, manufacturing orders, advertising campaigns, delivery systems, project management, and resource allocation.

## **14. When Should DP Be Used?**

Dynamic Programming is preferable when decisions depend on previous decisions, the problem contains overlapping subproblems, optimal substructure exists, and a locally best decision cannot always be guaranteed to be globally optimal.

## **15. When Should Greedy Be Used?**

Greedy is preferable when a locally optimal decision can be proven to lead to a globally optimal solution, especially when decisions can be made directly without maintaining many alternative states. It is usually simpler and faster when its greedy-choice property holds.

## **16. Key Insight: DP vs Greedy**

Do not choose an algorithm only because one is faster. First determine whether the problem has a valid greedy-choice property. Greedy is usually faster and simpler, but it is correct only when its local choices can be proven to lead to a global optimum. DP is more systematic for problems involving interacting decisions and overlapping subproblems.

## **17. Overall Analysis**

Assembly Line Scheduling demonstrates how DP finds the best path through sequential stations by comparing alternative line choices. Job Sequencing demonstrates how a Greedy approach selects high-profit jobs and places them in feasible deadline slots. They optimize different objectives and therefore require different correctness arguments.

## **Conclusion**

Assembly Line Scheduling and Job Sequencing illustrate the difference between Dynamic Programming and Greedy algorithms. DP is appropriate for Assembly Line Scheduling because each decision depends on previously computed states and alternative paths must be compared. Greedy is appropriate for the standard Job Sequencing with Deadlines problem because selecting the highest-profit job and placing it in the latest available slot is optimal under its assumptions. The main lesson is that algorithm selection depends on problem structure: DP should be used when decisions interact and overlapping subproblems exist, while Greedy should be preferred when a greedy-choice property and proof of optimality are available.